0计算机基础的Agent应用与开发入门 · 第 2 课

第二课：软件的组成——程序、数据、模块、职责与基本流程

> 说出一个软件由哪些东西组成；分清“程序”和“数据”的区别；把一个庞大的处理过程拆成若干个模块，并说清每个模块的职责；看懂程序里的顺序、分支和循环；画出一个小软件的结构图。

作者：藤氏蓝

> **本课你将学会：** 说出一个软件由哪些东西组成；分清“程序”和“数据”的区别；把一个庞大的处理过程拆成若干个模块，并说清每个模块的职责；看懂程序里的顺序、分支和循环；画出一个小软件的结构图。

一、先看一个你每天都在用的例子

打开手机地图，输入“附近的便利店”，屏幕上跳出一串店名，每个后面跟着距离和营业时间。你点第一家，它画出路线，告诉你走路要 12 分钟。

你做的事情只有两句话：搜一下，找一家。但软件在这几秒里做的事，多得多：它要先知道你站在哪里，才能算距离；再从几百万条商家信息里，找出名字里带“便利店”的那些；一条一条检查它们现在有没有在营业；把还开着的按距离从近到远排好；最后把结果画成你能看懂的列表。

这些事不是一口气做完的，而是拆成了好几块，每块专门负责一件事。这一课我们就打开这个软件，看看它到底由什么组成。

二、为什么要问“它由什么组成”

上一课我们把需求拆成了输入、处理、输出、数据。有一个问题当时留着了：处理这一步里写着“算距离”“筛出营业中的店”“排序”，听起来像是软件自己想清楚了，其实不是。

处理之所以能发生，是因为软件里真的写了这些规则。 规则写在什么地方、能不能被单独检查、换一个功能时会不会牵一发动全身，这些都取决于软件是怎么组织的。

这一点在和 Code Agent 打交道时尤其重要。当它写出两百行代码、跑起来报错了，你至少要能问出一个有意义的问题：是规则本身写错了，还是规则放错了位置？是模块之间的配合断了，还是数据本身就不对？不懂组成，你只能把报错原样贴回去，然后看它乱改一通。

三、四样东西，构成一个软件

组成	它是什么	一句话理解
程序（Program）	一串写下来的规则和步骤	照着做的指令
数据（Data）	程序要处理的东西	被加工的原料
模块（Module）	把大程序切成的一小块	各管一事的积木
职责（Responsibility）	这个模块负责什么、不负责什么	分工说明书

这四样东西的关系是：**程序是规则，数据是被规则处理的原料，程序被切成模块，每个模块承担一项职责。**

上一课我们说过“数据”是四个部分里最容易被忽略的。这一课要特别提醒另一件事：**程序和数据是两回事，它们是可以分开保存的。** 菜单价格是数据，“总价等于商品价格相加”是程序。改价不用动程序，改促销规则不用动数据。分清这两者，你在改需求的时候才知道该改哪儿。

四、逐一拆开看

1. 程序：一串非常死板的指令

程序没有直觉，也不会“看着办”。它就是一份步骤清单：先做这个，再做那个，做完第 3 步就跳到第 4 步。

打开任何一段代码，你看到的都是这件事。写程序的人要做的工作，是把一件含糊的事情，变成每一步都明确到不需要思考的指令。“算一下总价”对程序来说太含糊了，它需要的是：

1. 取出珍珠奶茶的单价；
2. 取出椰果的单价；
3. 把两个数相加；
4. 把相加的结果保存下来。

程序有一个特点值得记住：它只会严格按照写下来的顺序执行，不会跳过任何一步，也不会多做一步。 你以为它“应该懂你的意思”，其实它只懂写下来的那行字。

> **常见误区：** 把程序想成一个整体的东西。实际上软件里通常有很多个程序文件，一个负责算距离，一个负责读数据，一个负责画界面。它们被分开保存，装好之后一起工作。

2. 数据：被加工的原料

数据是程序处理的对象。在“找便利店”这件事里，数据包括：

- **商家信息**：店名、地址、经纬度、营业时间、有没有在营业；
- **你的位置**：手机定位或 GPS 得到的经纬度；
- **搜索词**：你输入的“便利店”这三个字；
- **筛选条件**：你勾选的“24 小时营业”“有停车位”；
- **算出来的结果**：距离 800 米、走路 12 分钟。

数据的特点是：它不负责做事，它只负责被查询、被比较、被计算。

一个判断数据的好办法： 如果把它的值换掉，程序的**做法**不变，只是**结果**变了，那它就是数据。比如把“24 小时营业”换成“8 点后仍营业”，程序该做的事完全没变，只是符合条件的店不一样了。

> **常见误区：** 把数据和显示它的界面混为一谈。界面上“800 米”这个数字是数据；它长得小一点、颜色淡一点，那是界面。修改需求时要说清楚改的是哪一个。

3. 模块：把大程序切成小积木

如果把所有规则都写在一个文件里，那个文件会长到没人敢改。所以程序员会把它们按功能切成一块一块，每一块叫做一个**模块**。

“找便利店”这件事，大致可以切成下面这些模块：

模块	它负责的事	它不负责的事
定位模块	知道你现在在哪里	不管你要找什么
搜索模块	按“便利店”这个词找到候选商家	不判断远近
数据模块	读出商家资料，告诉你有几家、最近的几家	不做筛选
筛选模块	留下正在营业的那几家	不计算距离
排序模块	按距离从近到远排好	不画界面
界面模块	把结果画成列表和地图	不重新算一遍

模块化的好处，可以用一句话概括：出了问题，你知道去问谁。

以前“顺序排错了”，你要在一大片代码里找；现在排列顺序是排序模块的职责，直接去看它。如果发现距离数值本身算错了，就应继续检查产生距离的数据或计算步骤。清楚的分工，让你可以区分“数据有问题”和“排序规则有问题”。

> **常见误区：** 按“界面”切模块。比如把整个软件切成“首页模块”“详情页模块”。这样切的问题是，同一段算价格的规则可能在两个模块里各写了一遍，改了一处另一处就忘了改。**按职责切，不要按界面切。**

4. 职责：分工说明书

职责是一个模块“该做”和“不该做”的清单。它是这个软件里最重要、也最容易被跳过的一句话。

上面那张表里，每一行的第二列写“负责什么”，第三列写“不负责什么”。第三列经常被跳过，但它是分工的关键。

判断职责是否清楚的三个问题：

1. 能不能用一句“只……不……”说清它？比如“只负责读商家资料，不负责筛选”。
2. 换一个功能需求时，这个模块需要改吗？如果不需要，它的职责就是干净的。
3. 它的名字能概括它做的事吗？“排序模块”能，“工具模块”不能。

> **一个小提醒：** 一个模块同时干三件事，通常意味着它应该被拆开。你在拆分的过程中，往往会发现原本想不清的需求——这是拆模块的额外收获。

5. 基本流程：程序怎么往前走

所有程序，往前走的方式只有三种。学会这三种，你看任何一段代码都知道它在干什么。

顺序：一件事做完，做下一件。 “读商家数据 → 算距离 → 排序 → 画出来”，从上到下，没有岔路。

分支：根据某个条件，往不同的方向走。 “如果这家店现在没营业，就跳过它；如果所有店都没营业，就提示‘附近没有营业中的便利店’。”

循环：同一件事重复做，重复到条件不成立为止。 “把候选商家一家一家拿出来检查营业状态”，检查完第一家还有第二家，检查完第二家还有第三家——这就是循环。

为什么循环特别重要？ 因为商家可能有一万家，你不可能把一万家店写成一万行代码。你写一个循环，让它替你重复。这就是“用规则代替重复”的思想，也正是第一课说的“处理”真正的样子。

把这三种方式画出来，就是下面这两张图。

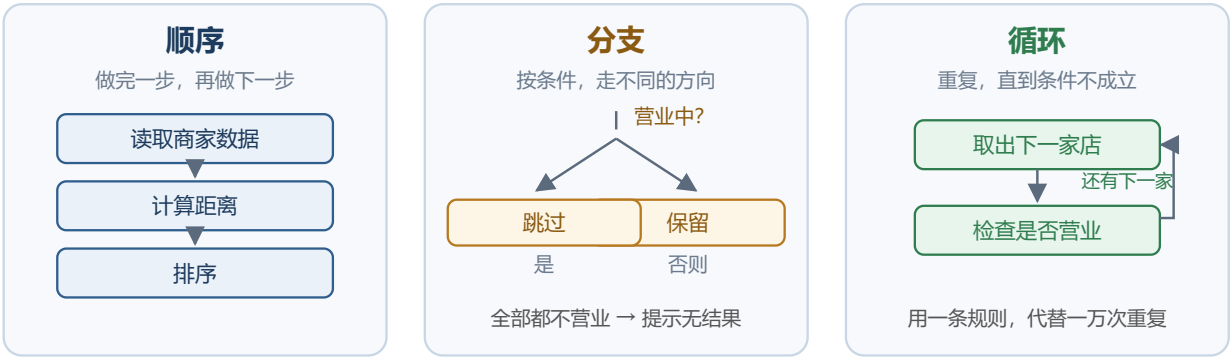


图 2-1　程序只有三种走法：顺序、分支、循环。所有复杂的程序，都是这三样反复组合出来的。

五、模块之间怎么配合

模块切好了，接下来要解决“怎么让它们互相找到对方”。这件事叫**调用**：一个模块做完某件事之后，把结果交给下一个模块。

“找便利店”这条链路是这样跑的：



图 2-2　模块的分工与配合。上方是四个模块各自的职责，下方是它们被调用的先后顺序；每个模块只回答自己那一个问题。

顺着箭头读一遍就清楚了：定位模块交出“你的位置”，数据模块交出“候选商家”，筛选模块交出“营业中的店”，排序模块交出“排好序的列表”，界面模块最后交出“你看到的那一屏”。

看懂这张图，你就具备了一种能力：拿到一个陌生软件，可以问它每一个箭头是什么。 出了问题，你能说出是哪一步断了，而不是笼统地说“它算错了”。

六、怎么画出一个软件的组成

不需要专业工具，一张纸就够了。按下面四步画：

1. **先画输入和输出。** 最左边的框是用户给的东西，最右边是用户看到的东西。这是你上一课已经会的。
2. **再切中间的“处理”。** 把处理按职责切成几个模块，每个模块一个框，名字用动词加对象，比如“排序”“筛选”，不要用“工具”“功能”。
3. **用箭头串起来。** 箭头方向就是数据流动的方向，旁边写清楚“这里传过去的是什么”。
4. **最后检查两个地方。** 有没有一个模块干了两件不相干的事？有没有哪个箭头没有写清楚传的是什么？

> **常见误区：** 画结构图时急着画界面。图上应该出现“输入”“筛选”“输出”这类字样，而不是“首页”“按钮”。画界面是第三、四课之后才做的事。

七、三个常见错误

错误一：把所有东西写在一起。 一个文件里塞进定位、筛选、排序的全过程。它能跑，但你无法检查，也无法安全地改。“能跑”和“能维护”是两回事。

错误二：模块名字起得太宽。 “工具模块”“公共模块”“管理模块”这类名字，等于没有分工。凡是起不出准确名字的模块，通常说明职责还没想清楚。

错误三：只写职责，不写“不负责什么”。 每个模块都写“负责处理数据”，等于所有人都负责，谁也不负责。分工的关键在边界，不在范围。

八、动手练习

这一节有三个由易到难的练习。建议先自己完成，再对照文末的参考答案。

练习一：分清程序和数据（热身）

下面是某记账 App 里的几句话。请把它们分进“程序”或“数据”两栏：

- 每笔记录包含日期、金额和分类；
- 每月 1 日自动生成上月报表；
- 咖啡 25 元；
- 支出超过 3000 元时显示提醒；
- 商家的默认分类列表。

分完之后再问自己两个问题： 如果把“咖啡 25 元”改成“咖啡 32 元”，需要改的地方属于哪一栏？如果把“每月 1 日生成报表”改成“每周一生成报表”，又是哪一栏？

类别	我分到的句子
程序	
数据	

练习二：给一个软件切模块

下面是一个“读书笔记 App”的需求，一共写了七条功能。请把它切成模块，每个模块只包含一件事，并在后面写出它的职责（用“只……不……”的句式）。

> ① 用户可以录入书名、作者和读到的页码；② 笔记可以关联到某本书的某一页；③ 首页按最近阅读时间展示书目；④ 可以给书打星；⑤ 笔记支持搜索；⑥ 搜索结果里要显示所在书名；⑦ 读完一本书后，想把它归档。

目标： 切出 4 到 7 个模块。如果有一个模块名称你一开始想不出好名字，就回头检查那一块的职责是否清楚；模块数量可以不同，关键是分工有依据。

模块名	它只负责……	它不负责……

练习三：画一张结构图

选一个你每天都在用的 App——可以是第一课练习一里拆过的那个，也可以换成别的。按第四节的四步画出它的组成，要求：

1. 至少画出 4 个模块；
2. 每个模块名是“动词 + 对象”；
3. 每条箭头上都标注了传过去的是什么；
4. 用到顺序、分支、循环三种走法的地方，各标出一处。

画完之后对照下面这份自检清单：

- [ ] 每个模块都能用“只……不……”说清职责；
- [ ] 每个模块的名字都能概括它做的事；
- [ ] 每条箭头都写清了传的是什么；
- [ ] 找不出一个模块同时干两件不相干的事；
- [ ] 一个从没见过这个 App 的人，能照着这张图说出它的完整流程。

挑战题（选做）

找一份现成的 App 的功能介绍（应用商店的截图说明就够），只凭这份文字，把它拆成模块和箭头。你会发现，介绍里写的是“功能”，而结构图上画的是“职责”，两者之间隔着一道翻译的功夫。这道功，就是你以后写任务说明时最需要的能力。

参考答案

练习一

类别	内容
程序	每月 1 日自动生成上月报表；支出超过 3000 元时显示提醒
数据	每笔记录包含日期、金额和分类；咖啡 25 元；商家的默认分类列表

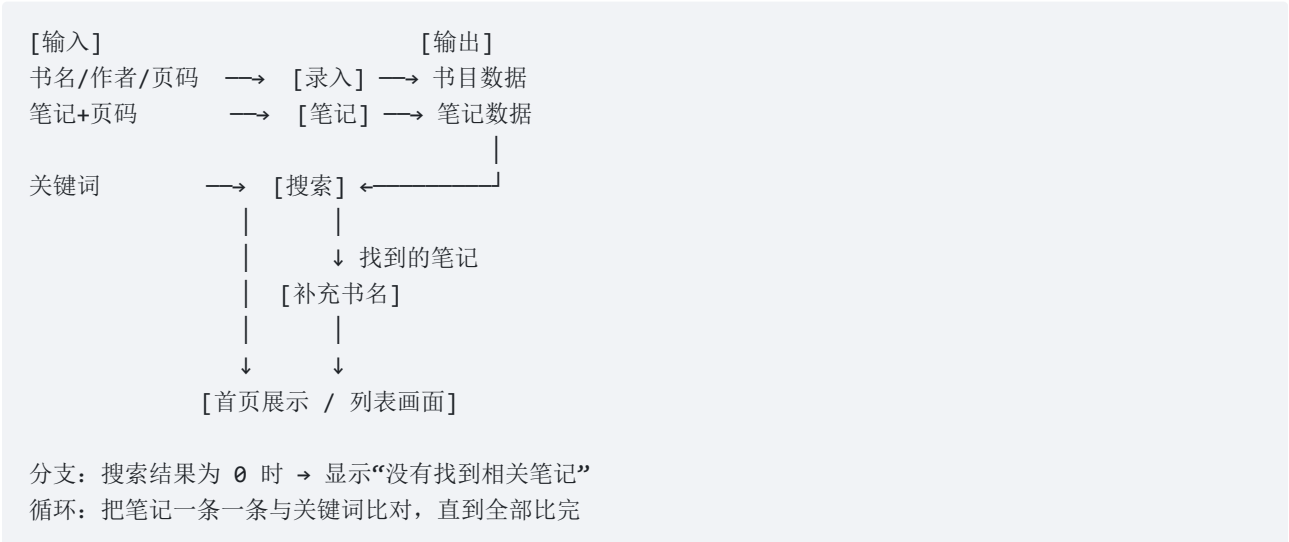
“咖啡 25 元”改成“咖啡 32 元”，改的是**数据**；“每月 1 日生成报表”改成“每周一生成报表”，改的是**程序**。这条规律很实用：改动落在哪一栏，决定了你该让 Code Agent 去动文件里的哪部分。

练习二（参考拆法，答案不唯一）

模块名	它只负责……	它不负责……
录入	只负责接收并保存书目信息	不负责展示，也不负责搜索
笔记	只负责保存笔记内容、页码和所属书	不负责判断笔记是否该显示在首页
首页展示	只负责决定首页显示哪些书	不负责去读笔记内容
打分	只负责记录和修改星级	不改变笔记本身
搜索	只负责按关键词找出匹配的笔记	不负责补上书名等附加信息
展示补充	只负责把搜索结果的书名显示出来	不负责找笔记
归档	只负责改变一本书的状态	不删除笔记

注意最后两行。第 ⑥ 条需求很容易被塞进“搜索”模块里，但它的职责是**展示**，和**查找**是两回事。硬塞进去，那个模块就同时干了两件事。想不出模块名的地方，往往就是需求里混着两种职责的地方。

练习三（参考示范：以“读书笔记 App”为例）



分支：搜索结果为 `0` 时 → 显示“没有找到相关笔记”
循环：把笔记一条一条与关键词比对，直到全部比完

对照自检清单：五个模块都写清了职责；“补充书名”单独成模块，正是因为它只做展示；每条箭头都标了传过去的东西；顺序、分支、循环各标出了一处。

检查点：学完这一课，你应该能……

- 说出一个软件由程序、数据、模块、职责这四样东西组成，并指出它们的关系；
- 判断一句话是程序还是数据，标准是“改它会不会改变程序的做法”；
- 把一段庞大的处理过程切成若干模块，每个模块用“只……不……”说清职责；
- 在一段陌生的描述里，找出顺序、分支和循环三种走法；
- 画出一张小软件的结构图，并说出每个箭头上传的是什么。

下一课预告

第三课《软件之间的连接》会接着追问：这些模块和数据全都在你的手机上吗？如果有一部分在另一台电脑上，手机怎样把请求发过去，又怎样拿到结果？

我们会继续用“找便利店”认识网络、客户端与服务端、请求与响应和 API。第四课《从设计到运行》再把前三课的图和说明连接到一个真正运行的小例子；第五课继续解释文件、环境、依赖与操作系统。体验版收录至第二课，第三课及后续内容以单课资料接续。